

Cesar Jackson - Week 5 Apache Flink Sessionization Grader Feedback

Course: DataExpert Boot Camp

Module: Apache Flink Sessionization

Submission: Week 5 Apache Flink Sessionization

Student: Cesar Jackson

Initial Grader Feedback

Hi Cesar - thanks for the thorough submission. You've clearly put care into the engineering of the Flink job, and your documentation shows you validated the core sessionization logic. Below is detailed feedback aligned to the rubric, plus a few concrete fixes/improvements to make your solution rock solid and fully compliant with the deliverables.

What's working well

- **Correct sessionization:** You key by (`ip, host`) and use a 5-minute Session window, with event-time and a watermark. Good filtering of blank/null identities.
- **Clean structure:** Clear separation of concerns (source, view, sinks, environment), and good use of `StatementSet` for multi-sink planning.
- **Sensible operational choices:** Checkpointing enabled, modest parallelism, environment variables for secrets, and nice run-time prints.
- **Results and reasoning:** You provided host-level averages and explained the offline validation limitations.

Gaps against the assignment deliverables

Missing SQL script The assignment asks for an SQL script to compute the metrics. Your job writes sessionized rows to Postgres (great), but you didn't include a separate `avg_session_events.sql`. Please add a simple script that:

- Computes the overall Tech Creator average (e.g., all hosts ending with `.techcreator.io`).
- Compares the three specified hosts.

Suggested `avg_session_events.sql`:

```
-- Average events per session for all Tech Creator subdomains
SELECT AVG(event_count)::numeric(10,4) AS avg_events_per_session_techcreator
FROM web_event_sessions
WHERE host LIKE '%.techcreator.io';
```

```
-- Average events per session by specified hosts
```

```

SELECT host,
       AVG(event_count)::numeric(10,4) AS avg_events_per_session
FROM web_event_sessions
WHERE host IN (
    'zachwilson.techcreator.io',
    'zachwilson.tech',
    'lulu.techcreator.io'
)
GROUP BY host
ORDER BY host;

```

Makefile target not provided Please add a Makefile target (`sessionization_job`) that starts your job in the course runtime. Example:

```

sessionization_job:
    docker compose exec jobmanager \
        flink run -py /opt/src/job/flink_sessionization_homework.py -d

```

If your repo uses a different path/layout, adjust accordingly and include brief run instructions in the README or HOMEWORK_ANSWERS.md.

JDBC sink for host summary needs a primary key (planner/upsert semantics) Your host summary sink is an unbounded aggregation (`GROUP BY host`) and will emit updates as new sessions arrive. The JDBC connector requires a primary key to handle upserts. Without a PK, planning or runtime can fail.

Fix: declare a (not enforced) primary key on `host` in the Flink DDL and create a physical table with a real `PRIMARY KEY` in Postgres.

Flink DDL change (host summary):

```

CREATE TABLE web_event_session_host_summary (
    host VARCHAR,
    session_count BIGINT,
    total_events BIGINT,
    avg_events_per_session DOUBLE,
    PRIMARY KEY (host) NOT ENFORCED
) WITH (
    'connector' = 'jdbc',
    'url' = '',
    'table-name' = 'web_event_session_host_summary',
    'username' = '',
    'password' = '',
    'driver' = 'org.postgresql.Driver'
);

```

Postgres physical table:

```
CREATE TABLE IF NOT EXISTS web_event_session_host_summary (
    host TEXT PRIMARY KEY,
    session_count BIGINT NOT NULL,
    total_events BIGINT NOT NULL,
    avg_events_per_session DOUBLE PRECISION NOT NULL
);
```

Physical table creation Flink's JDBC DDL registers a connector table, but it does not create the physical Postgres tables. Please include a small SQL DDL file that creates both target tables ahead of time:

```
-- sessions table (append-only)
CREATE TABLE IF NOT EXISTS web_event_sessions (
    ip TEXT NOT NULL,
    host TEXT NOT NULL,
    session_start TIMESTAMP(3) NOT NULL,
    session_end TIMESTAMP(3) NOT NULL,
    event_count BIGINT NOT NULL
);

-- host summary (upsertable)
CREATE TABLE IF NOT EXISTS web_event_session_host_summary (
    host TEXT PRIMARY KEY,
    session_count BIGINT NOT NULL,
    total_events BIGINT NOT NULL,
    avg_events_per_session DOUBLE PRECISION NOT NULL
);
```

Correctness and robustness notes

Event time parsing/time zone Your `TO_TIMESTAMP` with a literal `Z` parses the timestamp as local time, discarding the fact the source is UTC. This can shift timestamps if the JVM isn't set to UTC.

Options to fix:

- Explicitly pin the table local time zone to UTC at startup:

```
t_env.get_config().set("table.local-time-zone", "UTC")
```

- Or parse to epoch and then to `TIMESTAMP_LTZ` (works in Flink 1.16):

```
event_ts_ms AS UNIX_TIMESTAMP(
    event_time,
    'yyyy-MM-dd' 'T' 'HH:mm:ss.SSS' 'Z'
) * 1000,
event_timestamp AS TO_TIMESTAMP_LTZ(event_ts_ms, 3)
```

Either way, ensure your session window times truly reflect UTC input.

Watermark tolerance A 15-second watermark may be tight for out-of-order Kafka events. If you observe prematurely closed sessions, consider bumping this to 60-120 seconds.

Startup offsets You use `latest-offset`, which is fine for a live pipeline, but for initial backfills and QA, consider `earliest-offset` during validation runs.

Clarity of answers

You answered the per-host averages using the offline dataset and explained the limitations. Good.

The question “What is the average number of web events per session for a user on Tech Creator?” suggests an overall figure for the Tech Creator platform. Please add the “overall Tech Creator” average (e.g., `host LIKE '%.techcreator.io'`) as a single number alongside the per-host comparison.

Testing instructions to add

Prerequisites Confirm the JDBC and Kafka connector JARs plus Postgres driver are on the classpath of your Flink cluster.

Provide a `.env` or documentation listing required environment variables:

```
KAFKA_URL
KAFKA_TOPIC
KAFKA_GROUP
KAFKA_WEB_TRAFFIC_KEY
KAFKA_WEB_TRAFFIC_SECRET
POSTGRES_URL
POSTGRES_USER
POSTGRES_PASSWORD
```

Steps

1. Create Postgres tables (run the provided DDL SQL).
2. Run `make sessionization_job` (or the equivalent Docker Compose command).
3. Verify writes:

```
SELECT COUNT(*) FROM web_event_sessions;
```

4. Run `avg_session_events.sql` to produce the requested metrics.

Minor code polish suggestions

- Make watermark and checkpoint intervals configurable via environment variables.

- Consider adding a restart strategy to the `StreamExecutionEnvironment` for resilience.
- For the Kafka source, consider `'json.ignore-parse-errors' = 'true'` if you expect imperfect data.

Overall assessment

Strong Flink job and thoughtful engineering choices. You met the core sessionization requirement and wrote sessions to Postgres. The main gaps are deliverables (SQL script, Makefile target), host-summary sink upsert semantics (missing PK), and time zone parsing. Address those and this is an A-level submission.

If anything in my review environment assumptions doesn't match yours, please provide:

- Your Makefile and/or exact runtime command(s).
- `docker-compose.yml` (if used), especially how connector JARs are provisioned.
- The physical Postgres DDL you used (if any) and the database connection details format (e.g., `POSTGRES_URL`).
- A short sample of the raw Kafka `event_time` strings to double-check the parser/time zone.

Instructor's Feedback

Hi Ceasar - thanks for a very strong submission. You clearly put care into correctness, operational hardening, and documentation. Here's my review.

What's great

- **Correct sessionization:** You key by (`ip`, `host`) and use a 5-minute Session window with proper event-time semantics and a watermark. Good call pinning `table.local-time-zone` to UTC.
- **Clean Table API job:** Clear separation of concerns, good use of a `StatementSet` for the multi-sink plan, and explicit checkpointing.
- **PostgreSQL integration:** Append-only session sink and an upsertable host-summary sink with a primary key. Including `create_sessionization_tables.sql` is thoughtful and reduces reviewer friction.
- **SQL deliverables:** `avgsessionevents.sql` answers both required questions and returns nicely formatted averages.
- **Documentation and testing:** `HOMEWORK_ANSWERS.md` is thorough, includes validation evidence, constraints, runtime requirements, and a Makefile workflow description.

Correctness notes and minor suggestions

Event-time parsing Using `TO_TIMESTAMP` with a literal `Z` and pinning to UTC is sound. If you ever upgrade, consider `TO_TIMESTAMP_LTZ` for even clearer semantics.

Count logic Counting `ip` is fine because you filtered out `NULL`/blank IPs; counting rows (`count(1)`) would be equivalent and communicates intent.

Watermark tolerance 15 seconds is reasonable, but be aware that late events arriving beyond watermark can be dropped from the intended session. You already made this configurable; that's good. Document trade-offs and recommended values for live vs. backfill.

Host normalization Domain names are case-insensitive. Consider `lower(host)` in the session view and in the sink(s) to prevent accidental split keys.

JDBC sink parallelism For the host-summary upsert sink, consider setting `'sink.parallelism' = '1'` to avoid PK hot-spot upsert contention and to ensure deterministic ordering. For the append-only sessions sink, you can keep parallel for throughput, optionally with buffer flush tuning.

Indexing for analytics Add an index on `web_event_sessions(host, session_start)` for faster queries and time-bounded reports.

SQL review

Queries are correct, concise, and use appropriate casts to `NUMERIC(10,4)`. Nice job.

Consider adding `WHERE event_count > 0` as defensive filtering (shouldn't be needed, but harmless).

For ongoing analytics, you might add a time filter (e.g., `session_start >= ...`) to support period-based reporting.

Makefile and reproducibility

You refer to a Makefile with targets (`postgres-init`, `sessionization_job`, `session-metrics`), but the Makefile content wasn't included in the files I received here (only a hash in `SHA256SUMS`). I can't verify those targets or the Docker Compose wiring. Please include the Makefile content (and any `docker-compose.yml` it depends on) so I can validate the workflow end-to-end.

Answering the assignment questions

Average number of web events per session for a user on Tech Creator

Using your offline validation results, computing a weighted overall average across `techcreator.io` hosts yields approximately **1.7373 events/session**:

- Sessions: 2,521 (`zachwilson.techcreator.io`) and 1,503 (`lulu.techcreator.io`)
- Weighted average: approximately **1.7373**

You also provided a SQL to compute this in Postgres: `AVG(event_count)` over `host LIKE '%.techcreator.io'`. Please add the actual value observed from your live/graded run (not the fixture) directly in your answers section for completeness.

Comparison across hosts (offline validation dataset you provided)

- `zachwilson.techcreator.io`: **1.7211** average events/session (2,521 sessions)
- `zachwilson.tech`: **2.0000** average events/session (1 session; very small sample)
- `lulu.techcreator.io`: **1.7645** average events/session (1,503 sessions)

Operational polish opportunities

- Consider writing a small smoke-check SQL or `psql` command in the README that asserts both tables are receiving rows, and a sanity-check against late event drops (e.g., count recent sessions in the last 10 minutes).
- If your Kafka value schema changes are expected, you could consider a JSON schema registry setup; out of scope here, but worth noting for real deployments.

What I could not verify due to missing artifacts in this request

The Makefile target definitions and any Docker Compose setup. Please provide the Makefile (and compose file, if used) so I can run:

```
make postgres-init
make sessionization_job
make session-metrics
```

If you intended to include a sample `.env` or example environment variable file, that would also help reproducibility.

Overall assessment

Meets the core requirements: sessionization by IP+host with 5-minute gap, session rows written to Postgres, SQL to compute averages, and host comparisons. Code is clean, robust, and thoughtfully documented.

Two small improvements would make it airtight:

1. Provide the Makefile so the workflow is fully verifiable.
2. Add the explicit observed Tech Creator overall average from the real run into the answers section (you have the SQL; just include the value).

Grading system note

The following files were not recognized (unknown format): `Makefile` (`.makefile`).

For grading, supported formats are: documents as PDF or plain text (`.txt`, `.md`, `.rtf`); images as `.png`, `.jpg`, `.jpeg`, `.gif`, `.webp`. Submit as a `.zip` or a single image.